# AI-Lingo

System Deployment, Integration
and Additional Deliverables

**Sections 5 and 6**

Technology Stack • UML Diagrams • API & Service Documentation
Testing & Validation • Deployment Strategy

Prepared for the AI-Lingo language-learning application project.

# 5. System Deployment & Integration

## 5.1 Technology Stack

AI-Lingo is a mobile language-learning application designed to provide personalized, AI-powered language practice through interactive conversations and learning activities. The system uses the following technologies:

| Technology | Purpose |
|---|---|
| Flutter | Develops the cross-platform mobile application user interface. |
| Dart | The primary programming language used to build the application. |
| Bloc/Cubit | Manages application state and separates presentation logic from UI components. |
| Firebase Authentication | Handles user registration, login, and authentication. |
| Cloud Firestore | Stores user profiles, learning progress, conversation records where required, and vocabulary data. |
| Firebase AI Logic with Gemini | Integrates generative AI for interactive conversations, language feedback, grammar corrections, and vocabulary suggestions. |
| Hive | Provides local storage for vocabulary and selected learning data available on the device. |
| Git and GitHub | Support version control, source-code collaboration, and team integration. |

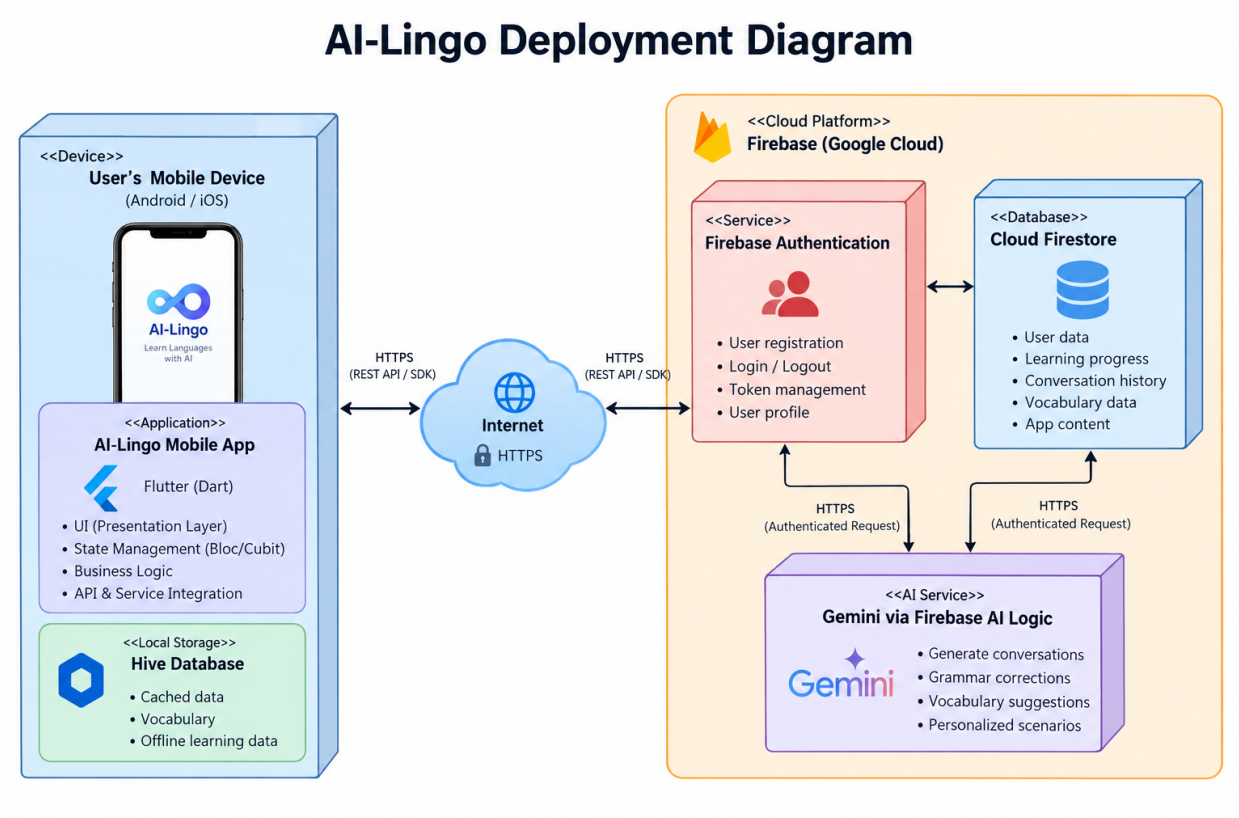
The application communicates with Firebase services and Gemini through supported integrations. Cloud Firestore provides cloud data storage, while Hive supports local persistence. The final documentation should reflect the services and packages actually configured in the project.

## 5.2 Deployment Diagram

The deployment diagram shows where application components run and how the mobile app communicates with cloud services and AI capabilities.

# 5.2 Deployment Diagram

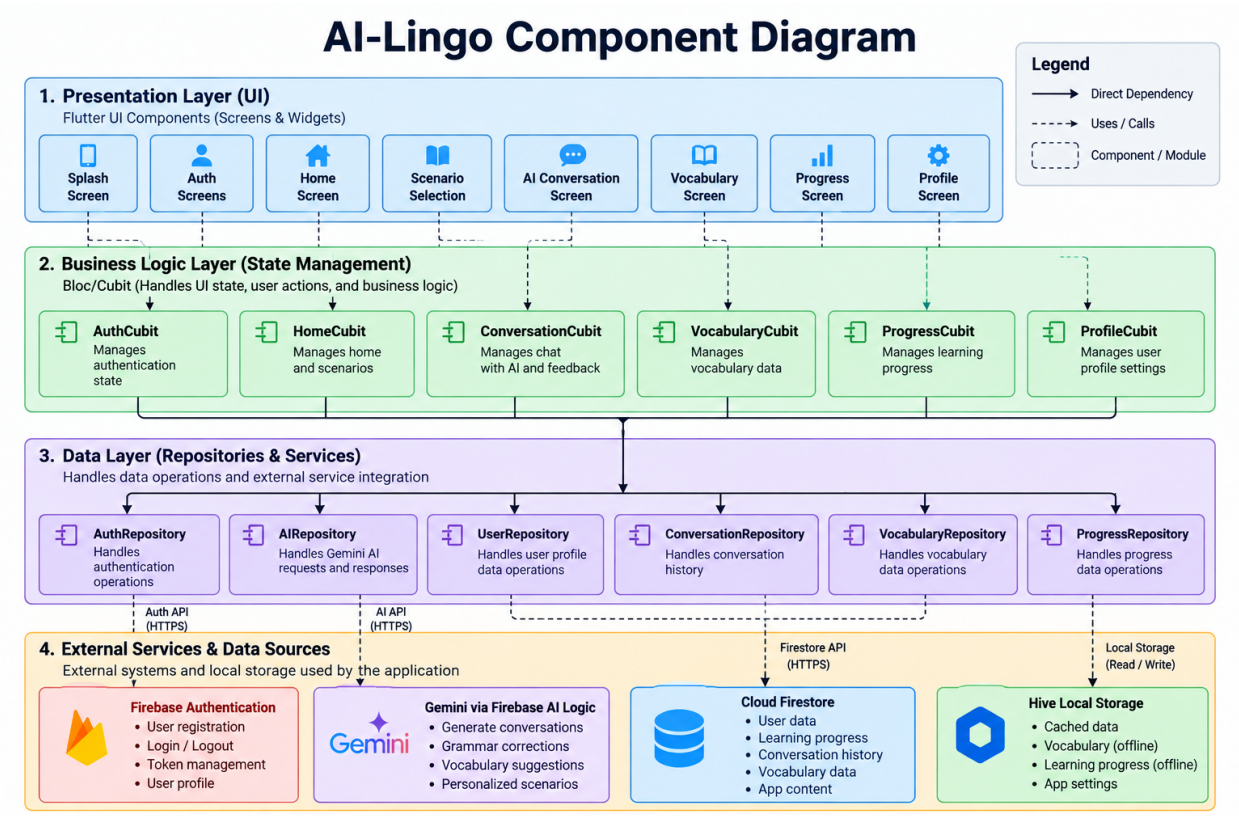
The diagram represents the intended deployment architecture: the Flutter mobile app runs on the user's device, Hive provides local storage, and cloud services support authentication, data persistence, and AI-generated learning content.




Note: Confirm the final integration flow and service configuration against the implemented application before submission.

# 5.3 Component Diagram

The component diagram presents the logical structure of the application, including the presentation layer, Bloc/Cubit state management, repositories and services, and external or local data sources.




Note: Repository and Cubit names are conceptual until verified against the actual codebase. Update the diagram if the implemented structure differs.

# 6. Additional Deliverables (if applicable)

## 6.1 API Documentation

AI-Lingo integrates with Firebase services and Gemini for authentication, cloud data management, and AI-powered language learning. It uses supported Firebase SDKs and Firebase AI Logic rather than relying on a separate custom backend API.

### 6.1.1 Firebase Authentication

- **Purpose:** Manage user authentication and account access.
- **Main operations:** Registration, login, logout, and authentication-state monitoring.
- **Input:** Credentials and account information required by the selected authentication method.
- **Output:** Authentication result, authenticated user identity when successful, or an error.

### 6.1.2 Cloud Firestore

- **Purpose:** Store and retrieve cloud-based application data.
- **Main operations:** Create or update user data, retrieve learning progress, and store or retrieve vocabulary and conversation records where implemented.
- **Input:** Document references and data fields defined by the Firestore data model.
- **Output:** Retrieved documents, successful writes, or errors when an operation fails or access is denied.
- Firestore access must be controlled through appropriately configured Firebase Security Rules.

### 6.1.3 Gemini via Firebase AI Logic

- **Purpose:** Provide AI-powered language-learning functionality.
- **Main operations:** Generate contextual conversations, provide grammar corrections and feedback, suggest vocabulary, and generate level-appropriate responses when supported by prompts and application logic.
- **Input:** Conversation context, selected scenario, relevant learning-level information, and the prompt for the requested AI task.
- **Output:** AI-generated conversation responses, corrections, or vocabulary suggestions.
- Handle loading states, invalid responses, network failures, and service errors.

### 6.1.4 Local Storage Using Hive

- **Purpose:** Store selected learning data locally on the user's device.
- **Main operations:** Save and retrieve cached vocabulary and preserve selected learning data for offline access when supported.
- Hive is local storage, not a remote API. Define what is stored locally and how it is synchronized if synchronization is implemented.

| Service | Integration Method | Main Purpose |
|---|---|---|
| Firebase Authentication | Firebase SDK | User authentication |
| Cloud Firestore | Firebase SDK | Cloud data storage and retrieval |
| Gemini | Firebase AI Logic | AI-generated learning content |
| Hive | Local database API | On-device data persistence |

Document exact SDK methods, data models, Firestore collection names, security rules, and error-handling behavior according to the final implementation. No custom REST endpoints are assumed in this design.

# 6.2 Testing & Validation

Testing verifies that AI-Lingo meets its functional and non-functional requirements. It covers individual components, integration between services, and complete user workflows.

## 6.2.1 Unit Testing

- Validate input fields and form-validation logic.
- Test Bloc/Cubit state transitions and error states.
- Test vocabulary processing and progress calculations.
- Test data-handling functions using controlled test data.

## 6.2.2 Integration Testing

- Verify registration and login through Firebase Authentication.
- Verify that user data and learning progress can be saved to and retrieved from Cloud Firestore.
- Verify AI conversation requests through Firebase AI Logic and handling of valid responses.
- Verify vocabulary data can be saved and retrieved through Hive.
- Verify error handling when a service is unavailable or a request fails.

## 6.2.3 User Acceptance Testing (UAT)

| Test Case | Expected Result |
|---|---|
| Register a new user | The account is created, or a clear error is displayed. |
| Log in with valid credentials | The user can access the application. |
| Select a learning scenario | The scenario is displayed and used in the learning session. |
| Send a message in an AI conversation | The response is displayed, or a clear error is shown. |
| Request grammar feedback | The application displays feedback from the AI workflow. |
| Save a vocabulary item | The item can be retrieved according to implemented storage behavior. |
| View learning progress | Available progress data is displayed correctly. |
| A service request fails | The failure is handled gracefully and feedback is provided. |

## 6.2.4 Non-Functional Validation

- **Performance:** Screens and common operations respond within acceptable time limits.
- **Reliability:** Errors and interrupted requests do not cause unexpected failures.
- **Security:** Authentication and database access rules protect user data.
- **Usability:** Navigation and learning activities are clear and understandable.
- **Compatibility:** Test supported mobile devices and operating system versions.

## 6.2.5 Test Results and Reporting

For each test, record the test identifier, preconditions, steps, expected result, actual result, status (Pass or Fail), and related defects. Mark a test as passed only after verifying the expected behavior.

# 6.3 Deployment Strategy

AI-Lingo is a cross-platform Flutter application. The deployment strategy covers service configuration, release preparation, validation, and distribution.

## 6.3.1 Deployment Environment

The application runs on supported Android and iOS devices. Firebase Authentication, Cloud Firestore, and Firebase AI Logic provide cloud functionality, while Hive supports local storage.

## 6.3.2 Deployment Steps

- **Configure the Firebase Project:** Register the app with Firebase and configure authentication, Firestore, and AI integration.
- **Configure Security:** Apply appropriate Firestore Security Rules and restrict access to user data.
- **Prepare the Application:** Verify app settings, dependencies, platform permissions, error handling, and configuration.
- **Run Tests:** Execute unit, integration, and end-to-end tests for the main learning workflows.
- **Build the Application:** Generate a release build. Android distribution may use an AAB for Google Play or an APK for testing.
- **Validate the Release Build:** Test authentication, data access, AI functionality, and local storage on supported devices.
- **Distribute the Application:** Share a test build with the team or evaluators; follow app-store requirements for public distribution.
- **Maintain the Application:** Monitor issues, fix defects, and release updates when needed.

## 6.3.3 Security and Configuration Considerations

Protect user data and service credentials. Firebase Security Rules should enforce the intended access permissions. Manage required configuration appropriately, and do not embed sensitive credentials in the application as unrestricted secrets.

The application should handle network interruptions, authentication failures, and AI service errors without exposing sensitive information.

## 6.3.4 Deployment and Scaling Considerations

Review Firebase and Gemini service usage, quotas, request limits, and applicable pricing before wider distribution. Consider response latency, error rates, database access patterns, and increased usage.

No separate application server or custom CI/CD pipeline is assumed in the current plan. These may be introduced later if required by the project's scope.

## 6.3.5 Deployment Outcome

The expected outcome is a tested and installable AI-Lingo mobile application configured to use the required Firebase services and Gemini integration, with appropriate data protection and error handling.

---

**Final note:** This document describes the intended architecture and test plan. Verify diagrams against the actual codebase and add real test results after execution.